Tensor Networ methods workshop I
julia langugage basics,
Exact Diagonalisation via XDiag.jl
Transverse Field Ising model and XXZ spin-1/2 model
entanglement entropy

October 21, 2026

Github with codes: `https://github.com/mkpanfil/TN_UW`

Source about `XDiag.jl`:`https://awietek.github.io/xdiag/user_guide/`

Exemplary IDE: `https://vscodium.com/`

Basic setup: Julia language, precompiling, multithreading

Please refer to the file
`https://github.com/mkpanfil/TN_UW/blob/main/julia_MKL_instructions.txt`

XDiag.jl working example

Check if you installed julia and XDiag.jl properly by running
`https://github.com/mkpanfil/TN_UW/blob/main/XDiag_free_fermions_example.jl`.

1 Transverse Field Ising model

The phase diagram of the TFI model looks like this:

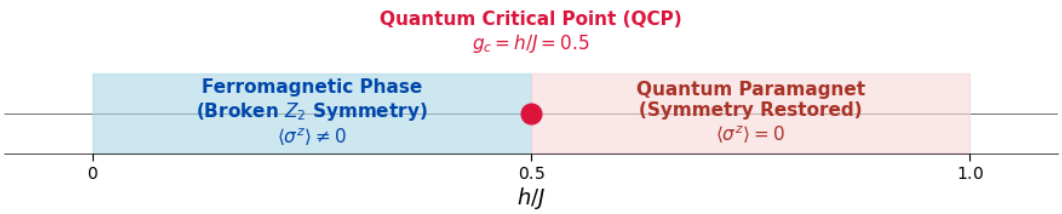


Figure 1: Transverse Field Ising phase diagram

The Hamiltonian of Transverse field Ising model is given by:

$$H_{\mathrm{TFI}} = -J \sum_{i=1}^{L-1} S_i^z S_{i+1}^z - h \sum_{i=1}^{L} S_i^x, \tag{1}$$

where J gives us strength of the interaction in the (z-direction) while h is the applied transverse field (x-direction) and S_i^x, S_i^z are spin-1/2 operators acting on site i, where we have sites going from site 1 up to L.

Spin-1/2 operators can be represented as 2x2 matrices:

$$S_x = \frac{\hbar}{2}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad S_y = \frac{\hbar}{2}\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad S_z = \frac{\hbar}{2}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Which are just Pauli matrices multiplied by $\hbar/2$:

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

We take $\hbar \to 1$.

$$H_{\text{OBC}} = -J(S_z \otimes S_z) - h(S_x \otimes I + I \otimes S_x)$$

For a smallest system with open boundary conditions (OBC) the number of sites $L = 2$

$$H_{\text{OBC}} = \begin{pmatrix} -J & -h & -h & 0 \\ -h & J & 0 & -h \\ -h & 0 & J & -h \\ 0 & -h & -h & -J \end{pmatrix}$$

Here the matrix is represented using "computational basis" which is z-resolved. $\{\uparrow\uparrow, \uparrow\downarrow, \downarrow\uparrow, \downarrow\downarrow, \}$.

You could try to diagonalize this matrix, but in overall it grows as 2^L, (where 2 is the size of the spin-1/2 operator) gets out of hand pretty fast. We need to use numerical methods for any serious system.

Methods for Sparse (Block-Diagonal) vs Dense matrices

The most general methods for finding eigenvalues are made for dense matrices (majority or all elements of matrix are nonzero).

$$H_{\text{TFI}}^{\text{OBC}}(L = 3) = \begin{pmatrix} -2J & -h & -h & 0 & -h & 0 & 0 & 0 \\ -h & 0 & 0 & -h & 0 & -h & 0 & 0 \\ -h & 0 & 2J & -h & 0 & 0 & -h & 0 \\ 0 & -h & -h & 0 & 0 & 0 & 0 & -h \\ -h & 0 & 0 & 0 & 0 & -h & -h & 0 \\ 0 & -h & 0 & 0 & -h & 2J & 0 & -h \\ 0 & 0 & -h & 0 & -h & 0 & 0 & -h \\ 0 & 0 & 0 & -h & 0 & -h & -h & -2J \end{pmatrix}$$

The matrix of TFI is actually block-diagonal, so its not exactly a dense matrix (easier to solve numerically).

1.1 Exercise: Find eigenvalues and eigenvectors of TFI matrix via XDiag.jl

Write the code to diagonalize the Ising Hamiltonian with $J = 1, h = 0$ to obtain ground state energy as well as first excited state energy, compute the gap. The functions to diagonalize the matrices are given on the XDiag.jl page: `https://awietek.github.io/xdiag/user_guide/`

The Hamiltonian **without the field** can be defined as:

```
block = Spinhalf(L) #We define the type and number of sites L here
ops = OpSum()

for i in 1:(L - 1)
    ops += -J * Op("SzSz", [i, i+1])
end
```

1.1.1 Save and Plot

Save the data and do a plot: Now do the same for $0 \leq h \leq 1$, plot the gap onto a plot, save plot and plot data to .csv file. Saving looks like this (you can also use dataframes):

```
energy_io = open("energy_data_L$(L).csv", "w")
println(energy_io, "h,E0,E1,Gap")
for (h, E0, E1, gap, c_val) in zip(h_range, E0_vals, E1_vals, gaps)
    println(energy_io, "$h,$E0,$E1,$gap")
end
close(energy_io)
```

While plotting looks like this:

```
plt_gap = plot(h_range, gaps,marker=:circle)
savefig(plt_gap, "TFIM_L$(L)_energy_gap.png")
```

1.1.2 Load and Plot

Load data from .csv and plot it (now for plotting you can also use python). Still in julia you can also just use:

```
energy_df = CSV.read("energy_data_L$(L).csv", DataFrame) #loads csv to dataframe
energy_df.h #this gives you list of fields
energy_df.Gap #this gives you list of gaps
```

1.1.3 Finite size scaling: fitting functions

Select some particular value of a field h, then for this field do energy E_{gap} computations for several system sizes, lets say $L = 4, 6, 8, 10, 12$, plot the E_{gap}/L against the $1/L$. To fit some simple polynomial you can use something like this:

```
using Polynomials
p_fit = fit(x, y, 2)
x_smooth = range(-3, 3, length=200)
y_fit = p_fit.(x_smooth)
```

2 XXZ model

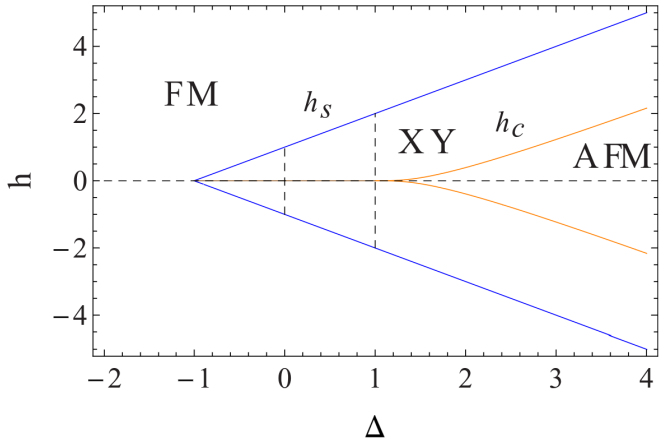


Figure 2: Parrallel field XXZ spin-1/2 model phase diagram
https://journals.aps.org/prb/pdf/10.1103/PhysRevB.93.054417

The Hamiltonian of the XXZ spin-1/2 model is given by:

$$H_{\text{XXZ}} = J(\sum_{i=1}^{L-1} S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + \Delta S_i^z S_{i+1}^z) - h\sum_{i=1}^{L} S_i^x, \tag{2}$$

or alternatively:

$$H_{\text{XXZ}} = J(\sum_{i=1}^{L-1} S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+ + \Delta S_i^z S_{i+1}^z) - h\sum_{i=1}^{L} S_i^x, \tag{3}$$

In XDiag.jl it is implemented as:

```
for i in 1:L-1
    ops += J *Op("Exchange", [i, i+1])
    ops += J *Delta*Op("SzSz", [i, i+1])
end
```

2.1 Exercise: 2D Phase Diagram (can be python also)

Start at $h = 0$ and find phase transitions by varying Δ. Do the same plots as in Transverse Field Ising case.

2.1.1 Measurement of the magnetic moment of a state

Info about measurements can be found here: https://awietek.github.io/xdiag/user_guide/09-measurements/
Then try to do the full phase diagram as seen in Figure 2.

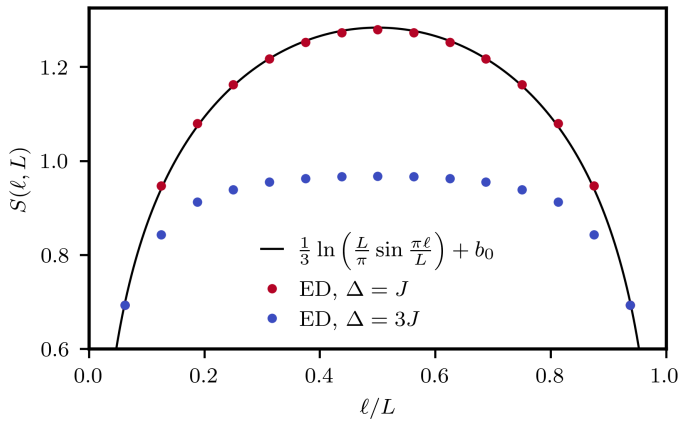


Figure 3: Entanglement entropy curve fitted with $S(l)$ function

3 Entanglement entropy measurement and central charge

3.1 Exercise: compute central charge for TFI model

I do not recommend code from ***https://awietek.github.io/xdiag/examples/entanglement_ground_state/****, it does not seem to work.*

Come back to the TFI model, implement computing eigenvectors and from eigenvectors compute entanglement entropy along the whole chain (can also be half a chain).

Fit the following function (given in here `https://arxiv.org/pdf/hep-th/0405152`)

$$S(\ell) = \frac{c}{6}\ln\left[\frac{2L}{\pi a}\sin\left(\frac{\pi\ell}{L}\right)\right] + s_0$$

Central charge here is given by c, you need to do fit also with s_0 parameter.

To simplify things you can obtain eigenvector in a simple vector form with LinearAlgebra library:

```
F = LinearAlgebra.eigen(Symmetric(Matrix(H_mat)))
psi0 = F.vectors[:, 1]
```

To perform SVD you can use:

```
LB = L - Subsystem
dimA = 2^Subsystem
dimB = 2^LB

psi_mat = reshape(psi, dimB, dimA)'
sv = LinearAlgebra.svdvals(psi_mat)
```

3.1.1

Finish the last verbatim section by using singular values `sv` to compute entanglement entropy.

3.1.2

Do the plot as in Figure 3.

3.1.3

For TFI do the central charge plot as a function of a field

3.2 Exercise: XXZ central charge

Do the central charge plot (2D) for the XXZ model. Where fit is of bad quality exclude the incorrectly computed values. Compare the plot with the magnetisation measurments

Author: *Jędrzej Wardyn*